

Speaker Credit History Leaks the Target in LIAR: A Leakage-Free Benchmark for Fine-Grained Fake News Detection

Aaron Koshy Parayil

Department of Computer Science, Christ University, Bengaluru 560073, India

E-mail: aaron12parayil@gmail.com

Abstract The LIAR benchmark ships five per-speaker “credit history” columns that count how often a speaker has been rated at each truthfulness level, and a large part of the published gain from metadata on this dataset is attributed to them. We show that these columns cannot be used as features as distributed. Measured over the full corpus, the count vector is constant for 3 314 of 3 318 speakers and equals that speaker’s label tally over the training, validation and test splits for 2 846 of 2 947 comparable speakers; 1 689 of 1 701 single-statement speakers carry a one-hot encoding of their own label. Two leaks follow: every statement is counted inside its own features, and held-out labels are visible during training. A random forest given only these five columns reaches 0.394 six-class test accuracy, and the obvious repair of subtracting the row’s own label makes matters worse, not better, reaching 0.659. We then rebuild speaker history under three rules — training labels only, out-of-fold encoding, and prior smoothing — which returns the history-only model to 0.228, just above the 0.208 majority-class rate. On this corrected footing we benchmark six classical classifiers over three feature modes and two task granularities under a protocol that selects on validation and scores the test split once. The best leakage-free configurations reach 0.285 six-class and 0.673 binary test accuracy, with the hybrid feature set beating both single-modality ablations on both tasks.

Keywords credit history, data leakage, fake news detection, LIAR dataset, text classification

1 Introduction

Misinformation travels faster than the institutions built to correct it. A single false claim can shape opinion long before a fact-check reaches the audience that saw it, and the volume of claims produced daily by social platforms, political speeches and news aggregators far exceeds what any team of human fact-checkers can verify. That gap between the speed of circulation and the speed of correction is what motivates automated verification.

Most early work framed the task as binary classification: a statement is either true or false, and the model sorts one from the other. The framing is convenient for dataset construction, but it does not match how political statements actually work. A speaker may state a fact correctly while omitting the context that changes its meaning; a claim may be accurate in one setting and misleading in another. Fact-checking organisations recognised this long ago, which is why PolitiFact rates statements on an ordinal scale — *true*, *mostly-true*, *half-true*, *barely-true*, *false* and *pants-fire* — rather than with a yes-or-no verdict. A system that collapses that scale is solving an easier and less useful version of the problem.

The LIAR dataset [1] was assembled specifically to keep the harder version in view. It pairs 12 836 short PolitiFact statements with their six-way verdicts and with speaker metadata: party, home state, job title, the subject tags of the claim, the venue in which it was made, and five integer columns summarising how often that speaker had previously been rated *barely-true*, *false*, *half-true*, *mostly-true* and *pants-fire*. Those last five columns — the *credit history* — are the dataset’s distinctive contribution. They encode exactly the kind of relational, extra-textual context that a bag of words cannot recover from a twenty-word sentence, and essentially every system that reports a metadata gain on LIAR consumes them in some form.

This paper starts from an uncomfortable observation

about those columns. They are not a summary of a speaker’s *past*. They are a summary of the speaker’s record across the *entire distributed corpus*, computed once and copied onto every row belonging to that speaker. The consequence is twofold. First, each statement is counted inside its own feature vector, so the label a model is asked to predict has already been added to the evidence it is given. Second, because the same vector is copied onto that speaker’s validation and test rows, held-out labels are visible at training time through a shared aggregate. Both are textbook target leakage in the sense of Kaufman *et al.* [2]: information about the target that is available to the model at training time but would not be available at prediction time.

The effect is not subtle. A random forest that sees nothing but the five credit history columns — no statement text, no party, nothing else — reaches 0.394 six-class test accuracy, well above the 0.274 reported for the hybrid convolutional model introduced with the dataset [1]. The obvious repair, subtracting one from the bucket corresponding to the row’s own label, does not fix the problem: it makes it far worse, lifting the same model to 0.659 accuracy. The reason is instructive and is analysed in Section 3.5. Because the per-speaker vector is constant and therefore memorisable, subtracting the row’s own label turns the feature vector into an almost injective encoding of that label.

Correcting this changes what the benchmark says. Once speaker history is rebuilt so that it uses training labels only, is computed out-of-fold so that no statement contributes to its own features, and is smoothed toward the corpus prior so that sparse speakers do not produce saturated vectors, the same history-only model falls to 0.228 — barely above the 0.208 majority-class rate. Speaker history remains a real but modest signal, roughly two accuracy points rather than nineteen.

1.1 Contributions

This paper makes the following contributions.

- **A quantified leakage audit of LIAR’s credit-history columns** (Section 3). We characterise the leak structurally — constancy per speaker, agreement with the full-corpus label tally, one-hot self-encoding for single-statement speakers — and measure its effect size with a history-only classifier. Every number is produced by a single script in the accompanying repository.
- **An analysis of why the natural repair fails** (Section 3.5). We show that self-exclusion alone is insufficient whenever the aggregate is constant per group, and state the condition under which subtracting the row’s own contribution amplifies rather than removes the leak.
- **A leakage-free speaker-history encoder** (Section 4.3) built on three rules — train-only label support, out-of-fold encoding, and prior-smoothed proportions — together with auxiliary features that let a model know how much evidence a given history estimate rests on.
- **A corrected benchmark** (Section 7) of six classical classifiers across three feature modes (text only, metadata only, hybrid) and two task granularities, under a protocol that performs all selection on the validation split and scores the test split exactly once. We report accuracy, macro-F1 and weighted-F1 with interval estimates, and we give per-class results and a confusion analysis for the strongest configuration.
- **Practical guidance** (Section 8.4) for future work on LIAR: what to do with the shipped columns, what a defensible metadata baseline looks like, and which published comparisons are and are not safe to make.

We deliberately restrict the model family to classical, fully reproducible classifiers. The point of the paper is the measurement protocol, not a new architecture; introducing a transformer or a speaker graph at the same time as the corrected features would make it impossible to attribute any change to the correction. Section 9 sets out how those extensions should consume the corrected history when they are added.

1.2 Organisation

Section 2 reviews related work across the paradigms that have been applied to LIAR and positions this paper among them. Section 3 describes the dataset and presents the leakage audit. Section 4 describes the corrected pipeline. Section 5 gives the experimental setup, Section 6 the exploratory analysis, and Section 7 the results. Section 8 discusses what the corrected numbers mean for the literature and states the limitations of the study. Section 9 concludes.

2 Related Work

2.1 Why Binary Framing Falls Short

Most early work in this area treated fake news detection as a two-class problem. The framing simplifies dataset

construction, but it does not survive contact with how fact-checkers operate: a statement is rarely simply true or false, and where it falls on the scale often depends on material outside the sentence — who said it, what they meant, and what was omitted. Producing that judgment is a different and considerably harder problem than binary classification, and it is the problem LIAR was designed to expose. The distinction matters when reading reported accuracies: binary numbers above 0.90 on corpora such as WELFake are routinely quoted alongside six-way numbers below 0.50 on LIAR, and the two are not measuring the same difficulty.

2.2 Feature-Based Models and Their Ceiling

LIAR was introduced to move the field past binary labels [1]: roughly 12 800 statements rated on a six-point scale, with speaker metadata attached. Standard models — support vector machines, logistic regression, and a simple convolutional network — reached around 0.27 six-way accuracy, which says less about those models than about the difficulty of the task once the label space is no longer binary. A follow-up approach supplied the fact-checkers’ written justifications alongside the statement and trained a bidirectional LSTM, reaching roughly 0.36 [3]. That is a substantial jump, but the justification text is written *after* the verdict, so a system that consumes it is not performing the same task as one that must decide from the claim and its context alone. This is worth naming explicitly, because it is the same category of problem as the one we document for the credit-history columns: a feature that encodes the answer produces an impressive number and a useless model. Elsewhere, a CNN-LSTM applied to Hindi-language news reported very high accuracy within that language [4], though transfer to other languages and domains was not established.

2.3 Modelling Speakers as a Network

A statement does not exist in isolation, and several lines of work have tried to capture who is speaking. One approach builds a graph over speakers, parties and statements and frames detection as node classification with a relational graph neural network [5], improving on text-only baselines to roughly 0.38 but degrading on speakers absent from the graph — a common deployment scenario, and one the LIAR splits themselves exhibit, since 15.6% of test statements come from speakers with no training-set history at all. A later approach combined a transformer with a graph convolutional network and contrastive learning, reporting close to 0.46 [6] at the cost of substantial graph-construction overhead. A co-attention GNN over user profiles reported much higher figures on GossipCop and PolitiFact [7], but those corpora are structured differently from LIAR and the numbers are not directly comparable.

2.4 Transformers

Once attention-based encoders became standard, several groups applied them directly. Fine-tuned BERT reached roughly 0.41 six-way accuracy on LIAR [8]; the 512-token

limit is not a binding constraint for statements averaging eighteen words, so the remaining difficulty is contextual rather than lexical. Folding metadata in through dual attention and an MLP fusion layer reached about 0.435 [9], with sensitivity to missing metadata fields — relevant here, since job title and state are absent for 27.9% and 21.5% of LIAR rows respectively.

2.5 Large Language Models and Efficient Fine-Tuning

As models grew, the question shifted from representation quality to adaptation cost. Prompt and prefix tuning on GPT-2 avoided full retraining [10] but proved sensitive to prompt wording. DeBERTa-v3 gave a further increase to roughly 0.47 [11] at higher compute cost. LLaMA-2 and Mistral were evaluated in few-shot settings without fine-tuning [12], with five-shot prompting reaching about 0.46 at high inference cost. The strongest reported results combine large pretrained encoders with efficient fine-tuning: around 0.48 six-way accuracy and close to 0.90 binary accuracy across BERT, RoBERTa and DeBERTa [13], a comparable gain from LoRA on Mistral-7B [14], and binary accuracy above 0.99 with lightweight adapters [15] — the last on a binary corpus, which as noted above is a different problem. A mixture-of-experts arrangement on LLaMA-3.1 that routes text and metadata through different experts reported roughly 0.49 [16], requiring multi-GPU infrastructure.

2.6 Generalisation, Explainability and Robustness

Beyond raw accuracy, three practical issues recur. Cross-dataset transfer is weak, and domain-adversarial training has been used to recover a cross-domain F1 near 0.74 [17]. Explainability has been approached with attribution methods layered over DeBERTa [18], which matters if such systems are to be trusted operationally. Robustness to deliberate manipulation remains open; adversarial training against GPT-4o-generated attacks held up only partially [19]. Self-supervised pretraining has been explored to reduce label dependence [20], and temporal drift — the degradation of a model as political discourse moves — has been studied directly [21], a failure mode invisible in a static benchmark.

2.7 Leakage and Evaluation Hygiene

The methodological literature this paper leans on is older and largely outside misinformation research. Kaufman *et al.* [2] give the standard formulation of leakage as the introduction of information about the target that would not legitimately be available at prediction time, together with the observation that leakage is usually discovered through implausibly good results rather than through code review. Out-of-fold construction of target-derived features, and smoothing of per-category statistics toward a global prior, are standard practice for high-cardinality categorical encoding [22]; the encoder in Section 4.3 is an application of that technique to a leak that happens to be baked into a published file rather than

Table 1. Reported six-class accuracy on LIAR across paradigms, with the reference points established in this paper

Approach	Acc.	Paradigm
Majority class (test)	0.208	trivial
Hybrid CNN [1]	0.274	CNN + metadata
BiLSTM + justifications [3]	0.360	post-hoc evidence
Relational GNN [5]	0.380	speaker graph
Fine-tuned BERT [8]	0.410	transformer
Metadata fusion [9]	0.435	transformer + meta
Graph-text joint [6]	0.460	graph + transformer
DeBERTa-v3 [11]	0.470	transformer
Transfer + PEFT [13]	0.480	LLM + PEFT
Mixture-of-experts [16]	0.490	LLM routing
<i>Reference points measured in this paper</i>		
Raw credit history only	0.394	leaking
Naive self-exclusion repair	0.659	leaking
Leak-free history only	0.228	corrected
Best leakage-free hybrid	0.285	corrected

Note: published figures are quoted as reported by their authors and were obtained under protocols that differ from ours; they are listed to locate the corrected numbers in the literature, not as a controlled comparison. The two rows marked **leaking** are measured here and are *not* valid performance estimates.

introduced by the modeller. On the evaluation side, Gorman and Bedrick [23] show that conclusions drawn from a single standard split are frequently not reproduced under resampling, and Dodge *et al.* [24] show that reported test numbers are uninterpretable without the size of the search that produced them. Both directly inform the protocol in Section 4.7.

2.8 Positioning

Table 1 summarises where this work sits. The lineage above is a story of increasing model capacity on a fixed benchmark. This paper asks a prior question — whether the benchmark’s metadata can be used at all as distributed — and finds that it cannot. The contribution is therefore not a competitive accuracy number; on the corrected footing no classical model is competitive with a fine-tuned encoder, and we do not claim otherwise. It is a corrected measurement protocol, a corrected feature construction, and a set of reference numbers that later systems can be compared against without inheriting the leak.

3 The LIAR Corpus and a Leakage Audit of Its Credit History

3.1 Corpus and Fields

LIAR [1] distributes 12 836 short statements collected from PolitiFact as three headerless tab-separated files. Each row carries fourteen fields: a statement identifier, the six-way verdict, the statement text, a comma-separated list of subject tags, the speaker’s name, job title, home state and party, five integer credit-history counts, and the venue or context in which the statement was made. Table 2 lists them with their cardinality and missingness over the combined corpus.

Table 2. Fields of the LIAR corpus, measured over all 12 836 rows

Field	Type	Distinct	Missing
id	identifier	12 836	0.00%
label	ordinal, 6 levels	6	0.00%
statement	free text	12 810	0.00%
subject	multi-label tags	144 tags	0.00%
speaker	categorical	3 318	0.00%
job_title	categorical	1 348	27.86%
state	categorical	70	21.46%
party	categorical	24	0.00%
context	categorical	5 080	1.00%
*_c (five columns)	integer counts	392 vectors	0.00%

Note: distinct values are counted after whitespace normalisation; missingness is measured before imputation. `subject` holds 4 547 distinct comma-separated combinations built from 144 atomic tags. The five credit-history columns are treated as a single vector here; only 392 distinct vectors occur across 3 318 speakers. The one-hot blocks of Table 7 use only the categories present in the training split (23 parties, 70 states).

Table 3. Label distribution across the official LIAR splits

Label	Train	Valid	Test	Corpus
<i>pants-fire</i>	842	116	92	1 050 (8.18%)
<i>false</i>	1 998	263	250	2 511 (19.56%)
<i>barely-true</i>	1 657	237	214	2 108 (16.42%)
<i>half-true</i>	2 123	248	267	2 638 (20.55%)
<i>mostly-true</i>	1 966	251	249	2 466 (19.21%)
<i>true</i>	1 683	169	211	2 063 (16.07%)
Total	10 269	1 284	1 283	12 836
Majority share	0.2068	0.2048	0.2081	0.2055
True-leaning share	0.5621	0.5202	0.5666	0.5584

Note: the true-leaning share is the positive rate of the binary task, in which *{half-true, mostly-true, true}* are collapsed into one class and the remaining three into the other.

The six verdicts form an ordinal scale, from *pants-fire* (a fabrication) through *false*, *barely-true*, *half-true* and *mostly-true* to *true*. We keep the dataset’s own split into 10 269 training, 1 284 validation and 1 283 test statements, both because it is the comparison point used by every published result on this corpus and because re-splitting would obscure the cross-split structure that the audit below is about. Table 3 gives the label distribution per split.

Five of the six classes occupy a narrow band between 16% and 21% of the corpus; only *pants-fire* is markedly rarer at 8.18%. Unlike many binary misinformation corpora, LIAR therefore does not demand aggressive resampling to prevent a model from collapsing onto a majority class — but the 0.2081 majority rate on the test split is also a reminder of how little headroom a six-way accuracy figure actually has.

3.2 What the Credit-History Columns Contain

The five credit-history columns are described in the dataset’s documentation as the speaker’s total credit history counts, excluding the current statement. We examine that claim directly. Write $\mathcal{L}$ for the six labels and $\mathcal{C} = \mathcal{L} \setminus \{\text{TRUE}\}$ for the five labels that actually have a column — there is no `true_c` bucket, an asymmetry that turns out to matter. For

statement i let $s(i)$ be its speaker, y_i its label, and $\mathbf{b}_i \in \mathbb{N}^5$ its shipped count vector. Let $\mathcal{D} = \mathcal{D}_{\text{tr}} \cup \mathcal{D}_{\text{va}} \cup \mathcal{D}_{\text{te}}$ be the full corpus.

Table 4 collects the notation used here and in Section 4.

Table 4. Notation

Symbol	Meaning
$\mathcal{L}, \mathcal{C}$	the six labels; the five with a column
$\mathcal{D}_{\text{tr}}$ etc.	training, validation, test splits
$s(i), y_i$	speaker and label of statement i
$\mathbf{e}_c$	indicator vector for label c
$\mathbf{b}_i$	shipped count vector of row i
$\mathbf{B}_s$	speaker s ’s corpus-wide tally
$\mathbf{n}_s, \mathbf{n}_s^{(k)}$	training tally of s ; its fold- k part
$\mathbf{c}_i, T_i$	rebuilt counts for row i ; their sum
$\hat{\mathbf{p}}, m$	training label prior; smoothing strength
π_i	smoothed history proportions, Eq. (3)
$\mathbf{w}$	ordinal truthfulness weights
K, F_k	number of folds; fold k

Note: $\mathbf{b}$ and $\mathbf{B}$ refer to the columns as distributed; $\mathbf{c}, \mathbf{n}$ and π to the reconstruction of Section 4.3.

Definition 1 (Self-inclusion) A feature map ϕ is self-including if $\phi(i)$ depends on y_i . Any such feature is unavailable at prediction time, since y_i is what is being predicted.

Definition 2 (Cross-split aggregation) A feature map is cross-split aggregating if $\phi(i)$ for a training row i depends on $\{y_j : j \in \mathcal{D}_{\text{va}} \cup \mathcal{D}_{\text{te}}\}$. Any such feature transports held-out labels into training.

The audit is implemented in `scripts/leakage_analysis.py` and reports the quantities in Table 5.

Three facts follow from the table.

The vector is a per-speaker constant. For 3 314 of 3 318 speakers, every row belonging to that speaker carries an identical count vector. The columns are therefore not a running, statement-by-statement history; they are a single aggregate broadcast onto every row.

The aggregate spans all three splits. For 2 846 of the 2 947 speakers where the comparison is well defined, the shipped vector equals that speaker’s label histogram computed over the training, validation and test files combined. A training row belonging to a speaker who also appears in the test split therefore carries, in its features, a summary that counts that speaker’s test labels.

The current statement is inside its own features. The clearest case is a speaker with exactly one statement in the corpus. Then the aggregate has a single unit in it, and that unit is the row’s own label.

Proposition 1 (Single-statement recovery) Let s have exactly one statement i in $\mathcal{D}$, and suppose $\mathbf{b}_i$ equals s ’s corpus tally. Then $\mathbf{b}_i = \mathbf{e}_{y_i}$ if $y_i \in \mathcal{C}$, and $\mathbf{b}_i = \mathbf{0}$ if $y_i = \text{TRUE}$. In either case y_i is a deterministic function of $\mathbf{b}_i$.

Empirically, 1 689 of 1 701 single-statement speakers with a non-true label carry exactly $\mathbf{e}_{y_i}$, and all 355 single-statement true speakers carry the zero vector. That is 2 044

Table 5. Structural audit of the shipped credit-history columns

Property	Measurement
Count vector is constant per speaker	3 314 / 3 318
Counts equal the speaker’s tally over train $\cup$ valid $\cup$ test	2 846 / 2 947
Single-statement speakers whose counts one-hot encode their own label	1 689 / 1 701
<i>true</i> -labelled single-statement speakers with an all-zero count vector	355 / 355
Distinct count vectors across the corpus	392

Note: the second row compares only the 2 947 speakers for whom the comparison is well defined; *true* statements are excluded from the tally because no `true_c` column exists.

of 2 056 rows — 16% of the corpus — whose label is readable straight off the feature vector. Note the second clause: because *true* has no bucket, the all-zero vector is not a neutral “no history” signal but a positive indicator of a *true* label among single-statement speakers, which is precisely the kind of encoding a gradient-boosted or tree-based learner will find immediately.

3.3 Two Worked Examples

The abstractions above are easier to see in two actual rows of the corpus.

Self-inclusion. The very first row of `train.tsv` belongs to speaker `wayne-bohac`, carries the label *false*, and has the count vector (0, 1, 0, 0, 0) in the column order (barely_true, false, half_true, mostly_true, pants_fire). This speaker has exactly one statement in the entire corpus. The single unit in the vector is that statement. A model reading the second coordinate of the first training row is reading its label, and no amount of model-side regularisation changes that.

Cross-split aggregation. Speaker `john-edwards` has fifteen statements: twelve in training, two in validation and one in test. Every one of those fifteen rows carries the identical vector (0, 2, 4, 4, 1), which is the speaker’s tally over the whole corpus. The training rows alone would give (0, 1, 4, 4, 0). The difference is exactly the two validation statements — one *false* and one *pants-fire* — whose labels are therefore visible, in aggregate, inside the features of twelve training rows. The test statement is labelled *true* and contributes nothing, because no `true_c` bucket exists; had it been labelled anything else, a test label would have been visible during training in the same way.

Neither example depends on an unusual speaker. The first is the first row of the file, and the second is an ordinary politician with an ordinary number of fact-checks.

3.4 Effect Size

Structure alone does not tell us how much accuracy the leak manufactures, so we measure it. A random forest (100 trees, maximum depth 25, seed 42) is given *only* the five count columns — no statement text, no party, no state — and evaluated on the six-class task. Table 6 and Fig. 1 report the result under four feature constructions.

Table 6. History-only random forest on the six-class task, under four constructions of the same underlying signal

Construction	Test acc.	Macro-F1
Raw LIAR counts (as shipped)	0.3944	0.4056
Naive repair $\mathbf{b} - \mathbf{e}_y$	0.6594	0.6633
Leak-free out-of-fold history	0.2284	0.2236
Majority class	0.2081	—

Note: the first two rows are diagnostics, not performance estimates. Neither construction could be computed at prediction time for an unseen statement.

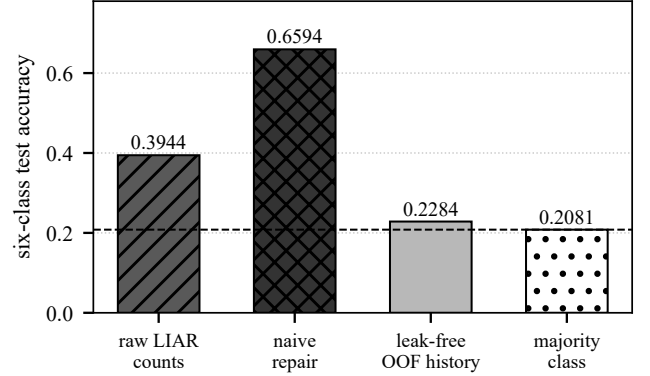


Fig. 1. Six-class test accuracy of a random forest given only speaker credit history, under four constructions of that history. The dashed line is the majority-class rate. The first two bars are leaking constructions and are shown to quantify the leak, not as performance estimates

Five columns of integers, with no access to what was actually said, reach 0.3944 — nearly double the majority rate, and well above the 0.274 originally reported for a hybrid convolutional model that used the statement text as well [1]. That gap is the signature Kaufman *et al.* [2] describe: a result too good for the information nominally available.

3.5 Why the Naive Repair Makes It Worse

The obvious correction is to remove the current statement’s own contribution: replace $\mathbf{b}_i$ with $\mathbf{b}'_i = \max(\mathbf{b}_i - \mathbf{e}_{y_i}, \mathbf{0})$ for $y_i \in \mathcal{C}$, leaving *true* rows untouched. The second row of Table 6 shows what happens: accuracy rises from 0.3944 to 0.6594. The repair does not remove the leak, it sharpens it.

The mechanism is worth stating precisely, because the same trap recurs whenever a group-level aggregate is self-inclusive.

Proposition 2 (Self-exclusion is invertible) *Suppose the aggregate $\mathbf{B}_s$ is constant across all rows of speaker s and observable from the training split, and let $\mathbf{b}'_i = \mathbf{B}_{s(i)} - \mathbf{e}_{y_i}$. If $\mathbf{B}_{s(i)}$ can be recovered from $\mathbf{b}'_i$ — in particular whenever $\mathbf{b}'_i + \mathbf{e}_c$ is an observed aggregate for exactly one $c \in \mathcal{C}$ — then y_i is determined by $\mathbf{b}'_i$.*

In other words, subtracting the label makes the feature vector a *displacement* from a memorisable landmark, and the direction of the displacement is the label. The premise holds here: there are only 392 distinct count vectors across 3 318 speakers, all of them observable in the training split, so the

landmark set is small enough for a tree ensemble to memorise outright. Measuring the consequence directly on the test split, the repaired vector determines the label exactly for 464 of 1 283 rows (36.2%), and the mean number of labels consistent with a repaired vector is 3.61 out of six. Exact recovery on a third of the split, plus a narrowed candidate set on much of the remainder, is enough to explain why a model that should have been weakened by the repair is instead twice as accurate.

The practical lesson generalises beyond LIAR: *self-exclusion is not sufficient when the aggregate is constant within a group*. The row's own contribution must be removed at a granularity coarser than the row — a fold, in the construction of Section 4.3 — so that the residual does not point back at the removed item.

3.6 Diagnosis

The credit-history columns fail both definitions above at once. They are self-including, so a model is shown the answer; and they are cross-split aggregating, so a model is shown the held-out answers too. Any published result that consumes them as distributed is measuring, to an unknown extent, a model's ability to read its own labels. We are not in a position to say which published systems are affected: papers reporting metadata gains on LIAR rarely state how these columns were handled, and the effect depends on whether they were fed in raw, aggregated, or dropped. What we can say is that the columns are unusable without reconstruction, that reconstruction must be more careful than subtracting the row's own label, and that the corrected signal is far weaker than the raw columns suggest — roughly two accuracy points over the majority class rather than nineteen.

The rest of this paper takes that correction as its starting point.

4 Method

4.1 Overview

Fig. 2 shows the pipeline. It is deliberately asymmetric across splits: every step that consumes labels is fitted on the training split alone, and the one step that consumes labels *and* is applied back to the training split — speaker history — is applied out-of-fold. Everything upstream of that is split-local, meaning it can be computed for a single row in isolation and therefore cannot leak by construction.

4.2 Preprocessing

Loading. The three files are read with no header and with quoting disabled, since statement text contains unbalanced quotation marks that a naive parser silently merges into neighbouring records. Each split is tagged with its origin so that the partition can be recovered after any concatenation.

Cleaning. Every object column is stripped of leading and trailing whitespace, and the strings "", "nan" and "None" are normalised to true missing markers. This matters more

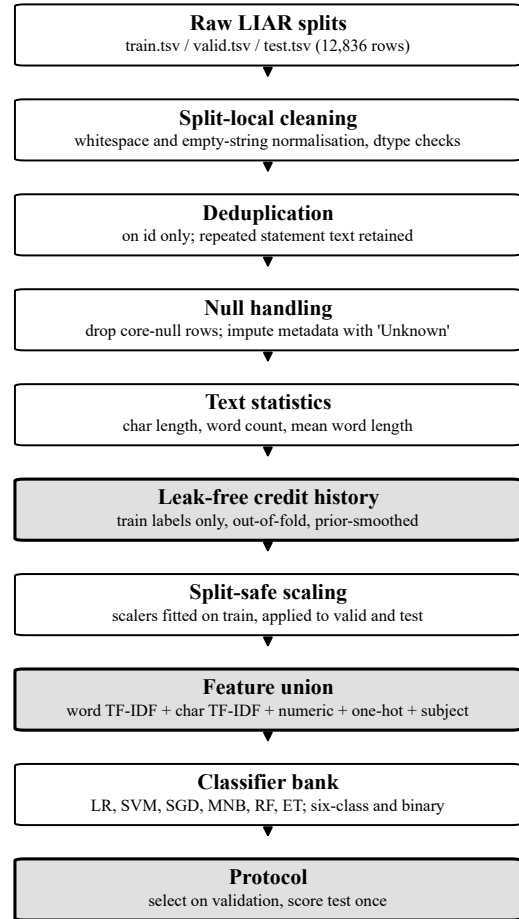


Fig.2. End-to-end pipeline. Shaded blocks are the three stages where the design differs from a conventional LIAR pipeline: label-derived speaker features are built out-of-fold from training labels only, the feature union excludes the shipped count columns, and all selection happens on the validation split

than it appears: an empty string is not missing as far as pandas is concerned, so omitting the step causes every subsequent missingness count to be wrong.

Deduplication. Duplicates are removed on id only. There are none. Twenty-six statements share their text with another row, and these are retained: politicians repeat talking points across appearances, and those are distinct fact-checked records, not database duplicates.

Null handling. Rows missing a core field (label, statement, party) are dropped; none are. High-missingness metadata (job_title, state, context, subject, speaker) is imputed with an explicit "Unknown" category rather than discarded, so that a row with a usable statement and label is not lost because one metadata field is absent. Treating absence as its own category also lets a model learn from the absence itself, which is informative here: missing job title and state correlate with speakers who are organisations, media outlets or social-media posts rather than individual officeholders.

Text statistics. Three surface features are computed per statement: character length, word count, and mean word length (characters divided by words). These are split-local

and cheap, and Section 6 shows they are close to uncorrelated with speaker history, so they contribute independent signal.

Label encoding. The six-way target is mapped to $\{0, \dots, 5\}$ in order of increasing truthfulness. The binary target collapses $\{\text{half-true}, \text{mostly-true}, \text{true}\}$ against $\{\text{pants-fire}, \text{false}, \text{barely-true}\}$, giving a 0.5666 majority rate on test. We state this explicitly because the binary collapse is not standardised in the literature: a two-way split at *mostly-true* instead yields a different majority rate and different numbers, and the two are frequently quoted against each other.

4.3 Leakage-Free Speaker Credit History

The shipped columns are discarded entirely for modelling purposes — they are retained in the processed files only so that the audit of Section 3 can be reproduced — and speaker history is rebuilt from scratch under three rules.

1. **Train-only support.** Only training labels ever enter any feature, so validation and test labels are never observed, directly or in aggregate.
2. **Out-of-fold encoding.** Training rows are encoded from the training rows *outside their own fold*, so no statement contributes to its own features, and — by Proposition 2 — the residual does not point back at what was removed.
3. **Prior smoothing.** Counts become proportions shrunk toward the global training prior, so a speaker with one prior statement does not produce a saturated, memorisable vector.

Formally, let $\hat{\mathbf{p}} \in \Delta^5$ be the training label distribution over all six labels — note that, unlike the shipped columns, *true* gets a bucket here — and let

$$\mathbf{n}_s = \sum_{j \in \mathcal{D}_{\text{tr}}, s(j)=s} \mathbf{e}_{y_j} \quad (1)$$

be speaker s ’s training label histogram. Partition $\mathcal{D}_{\text{tr}}$ into $K = 5$ stratified folds $F_1, \dots, F_K$ and write $\mathbf{n}_s^{(k)}$ for the histogram restricted to fold k . The count vector assigned to row i is

$$\mathbf{c}_i = \begin{cases} \max(\mathbf{n}_{s(i)} - \mathbf{n}_{s(i)}^{(k(i))}, \mathbf{0}), & i \in \mathcal{D}_{\text{tr}}, \\ \mathbf{n}_{s(i)}, & i \notin \mathcal{D}_{\text{tr}}, \end{cases} \quad (2)$$

where $k(i)$ is the fold containing i . With $T_i = \|\mathbf{c}_i\|_1$ and smoothing strength $m = 10$, the history features are the shrunken proportions

$$\boldsymbol{\pi}_i = \frac{\mathbf{c}_i + m \hat{\mathbf{p}}}{T_i + m}, \quad (3)$$

which is the standard smoothed target encoding for high-cardinality categorical variables [22]. Equation (3) interpolates between the speaker’s own record and the corpus prior as a function of how much evidence there is: at $T_i = 0$ it returns the prior exactly, and it reaches the halfway point at $T_i = m = 10$ prior statements.

Three auxiliary features accompany the proportions, all of which describe the *reliability* of the estimate rather than

Algorithm 1 Leakage-free speaker credit history

Require: training split $\mathcal{D}_{\text{tr}}$, target split $\mathcal{D}$, folds K , smoothing m

- 1: $\hat{\mathbf{p}} \leftarrow$ label distribution of $\mathcal{D}_{\text{tr}}$
- 2: $\mathbf{n}_s \leftarrow$ label histogram of speaker s over $\mathcal{D}_{\text{tr}}$, for all s
- 3: **if** $\mathcal{D} = \mathcal{D}_{\text{tr}}$ **then**
- 4: assign stratified folds $F_1, \dots, F_K$ over $\mathcal{D}_{\text{tr}}$
- 5: **for** $k = 1$ **to** K **do**
- 6: $\mathbf{n}_s^{(k)} \leftarrow$ histogram of s restricted to F_k
- 7: **for** $i \in F_k$ **do**
- 8: $\mathbf{c}_i \leftarrow \max(\mathbf{n}_{s(i)} - \mathbf{n}_{s(i)}^{(k)}, \mathbf{0})$
- 9: **end for**
- 10: **end for**
- 11: **else**
- 12: **for** $i \in \mathcal{D}$ **do**
- 13: $\mathbf{c}_i \leftarrow \mathbf{n}_{s(i)}$ ($\mathbf{0}$ if $s(i)$ unseen)
- 14: **end for**
- 15: **end if**
- 16: **for** $i \in \mathcal{D}$ **do**
- 17: $T_i \leftarrow \|\mathbf{c}_i\|_1$
- 18: $\boldsymbol{\pi}_i \leftarrow (\mathbf{c}_i + m \hat{\mathbf{p}}) / (T_i + m)$
- 19: emit $\boldsymbol{\pi}_i, \log(1 + T_i), \mathbf{1}[T_i = 0], \boldsymbol{\pi}_i^\top \mathbf{w}$
- 20: **end for**

its content: the support T_i enters as $\log(1 + T_i)$, an indicator $\mathbf{1}[T_i = 0]$ marks speakers with no usable history, and a single ordinal summary

$$\text{cred}_i = \boldsymbol{\pi}_i^\top \mathbf{w}, \quad \mathbf{w} = (-3, -2, -1, 0.5, 1, 2)^\top \quad (4)$$

collapses the distribution into a scalar in $[-3, 2]$ ordered from fabrication to truth. The indicator matters: 15.6% of test statements and 17.2% of validation statements come from speakers with no training history at all, and without the flag a model cannot distinguish “this speaker has a prior record centred on the corpus average” from “we know nothing about this speaker”.

Algorithm 1 states the procedure. Speaker keys are low-ercased and trimmed, and the placeholder values `unknown`, `nan`, `none` and the empty string are mapped to the null key so that unattributed statements are not pooled into one artificial mega-speaker.

Proposition 3 (Leakage-freeness of the encoder) *For $i \in \mathcal{D}_{\text{tr}}$, $\mathbf{c}_i$ is a function of $\{y_j : j \in \mathcal{D}_{\text{tr}} \setminus F_{k(i)}\}$, a set that excludes i . For $i \notin \mathcal{D}_{\text{tr}}$, $\mathbf{c}_i$ is a function of $\{y_j : j \in \mathcal{D}_{\text{tr}}\}$, a set that also excludes i . In neither case does any validation or test label appear.*

Two honest caveats follow from the construction. First, training rows see roughly $(K - 1)/K = 80\%$ of the available history while held-out rows see all of it, a mild distribution shift that the support feature $\log(1 + T_i)$ is there to expose. Second, a training row’s features do depend on the labels of *other* statements by the same speaker; that is the intended signal, not a leak, and it is exactly what a deployed system would have — a record of what the speaker has previously been rated.

Table 7. Feature blocks and their dimensionality under the default configuration

Block	Construction	Dim.
Word TF-IDF	1–2 grams, sublinear tf, min_df 2, max_df 0.95, English stop words	5 000
Char TF-IDF	char_wb 3–5 grams, min_df 3	20 000 [†]
Numeric	6 history proportions, cred., log support, unseen flag, 3 text statistics (z-scored)	12
One-hot	party (23), state (70)	93
Subject	TF-IDF over comma-split tags	140
Total, hybrid mode (default)		5 245

Note: [†] character n-grams are off in the default configuration and are treated as a binary choice during model selection (Section 4.7). The one-hot block grows to 4 000–9 000 columns when speaker, job_title and context are enabled.

4.4 Feature Representation

Five feature blocks are concatenated into a single sparse matrix. Table 7 lists them.

Word TF-IDF over unigrams and bigrams with sublinear term-frequency scaling [25], a vocabulary capped at 5 000, a minimum document frequency of 2 and a maximum of 0.95, with English stop words removed.

Character TF-IDF over 3- to 5-grams within word boundaries, capped at 20 000 features. Character n-grams are robust to the morphological variation and proper-noun density of political speech, and the search in Section 7.4 selects them for most configurations.

Numeric features, z-scored with a scaler fitted on the training split and applied unchanged to the held-out splits. Min–max variants are also computed and stored, so that a downstream model with different requirements can use them without re-deriving the pipeline.

One-hot metadata for categorical fields, with unseen categories at transform time mapped to all-zero rather than raising — necessary because the test split contains states and parties absent from training.

Subject tags, which are multi-label in LIAR ("energy, history, jobs"). Treating each combination as a category would produce 4 547 mostly-singleton levels, so the field is tokenised on commas and TF-IDF weighted over the 144 atomic tags, which keeps it dense enough to be useful.

Speaker identity deserves a note, because it might look like the same kind of shortcut as the credit-history columns. It is not. The speaker’s name is observable at prediction time, and one-hot encoding it lets a linear model learn a per-speaker intercept from training rows only — the same information that the history features carry in aggregated form, expressed as an identity. Where it does become a shortcut is in combination with the shipped counts, which is precisely the combination we have removed.

4.5 Feature Modes and Metadata Variants

Three feature modes isolate where the signal lives: *text only* (word, and optionally character, TF-IDF); *metadata only* (numeric, one-hot and subject blocks); and *hybrid* (all blocks). The comparison is only meaningful once the history features are leak-free — with the shipped columns, “metadata only” would be measuring the leak.

Within the metadata blocks, four nested variants are treated as a modelling choice rather than fixed: **base** uses party and state; **speaker** adds speaker identity; **speaker_subject** adds the subject tags; and **all_meta** adds job title and context. Combined with the binary character-n-gram switch, this defines the search space of Section 4.7.

4.6 Classifiers

Six classical classifiers span the usual families. All are `scikit-learn` implementations [26] with a fixed seed.

Multinomial logistic regression minimises the regularised cross-entropy $\sum_i -\log \hat{p}(y_i | \mathbf{x}_i) + \frac{1}{2C} \|\mathbf{W}\|_F^2$ with L-BFGS, giving calibrated posteriors over the six classes.

Linear SVM minimises the squared hinge loss in a one-versus-rest arrangement [27]; scores are converted to pseudo-posteriors by a softmax over decision values when probabilities are needed.

SGD classifier optimises the same family of linear objectives by stochastic gradient descent under a modified-Huber or log loss, included because it behaves differently from batch solvers on very sparse high-dimensional input.

Multinomial naive Bayes [28] models each class as a multinomial over feature counts with Laplace smoothing. It requires non-negative input, so the pipeline clips negative entries to zero before fitting. This is worth stating plainly: under naive Bayes the z-scored numeric features are effectively rectified, so the model sees only the above-average half of each history proportion. It is nonetheless the strongest hybrid model in our results, which we take as evidence that the coarse direction of the history signal, rather than its precise magnitude, is what carries.

Random forest [29] and **extremely randomised trees** [30] are included as the non-linear members of the bank, and because tree ensembles are exactly the models that exploit the leak of Section 3 most aggressively — a useful property for a diagnostic and a liability for a benchmark.

4.7 Selection Protocol

Reported test numbers are uninterpretable without knowing how much searching produced them [24], so the protocol is fixed in advance and stated here in full.

1. The search space is the product of feature mode, metadata variant, character-n-gram switch, and per-model hyperparameter grid: 476 configurations per task (Table 8).
2. Every configuration is fitted on the training split and scored on the *validation* split. The test split is not read at this stage.

3. The best configuration per (feature mode, model) cell by validation accuracy is carried forward — 18 per task.
4. Those 18 are refitted on the training split and scored once on the test split. No configuration is revisited afterwards.

Validation accuracy is reported alongside test accuracy in Section 7.4 only to document the protocol. It is optimistically biased by the size of the search and should not be read as a performance estimate; the gap between the two columns is itself informative, and we discuss it in Section 8.2.

Separately from the search, a *default configuration* — the fixed hyperparameters in `configs/baseline_config.yaml`, party and state as the only one-hot fields, no character n-grams — is run across all six (task, feature mode) cells. This is the ablation grid of Sections 7.2 and 7.3. It involves no selection at all, so its test numbers carry no selection bias, at the cost of not being tuned.

4.8 Metrics

Let C be the number of classes and TP_c, FP_c, FN_c the per-class counts. We report accuracy, the unweighted mean of per-class F1,

$$F1_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C \frac{2 TP_c}{2 TP_c + FP_c + FN_c}, \quad (5)$$

and the support-weighted mean of the same quantity. Macro-F1 is computed over all classes on both tasks, so a column labelled “macro” means the same thing everywhere; the positive-class F1 of the binary task is reported separately rather than overwriting the macro column. This is a small point of hygiene, but binary macro-F1 and positive-class F1 differ by up to eight points in our results and are easy to confuse across papers.

Because the test split has $n = 1283$ rows, an accuracy estimate carries a 95% Wilson interval [31] of roughly ± 0.025 at $p \approx 0.28$ and ± 0.026 at $p \approx 0.65$. Differences smaller than about two and a half points are therefore not resolvable on this split, and we avoid claiming them. This is a constraint of the benchmark, not of the models: LIAR’s test split is simply not large enough to separate systems that differ by one point, which is worth remembering when reading a leaderboard whose entries are spaced that closely.

4.9 Implementation and Reproducibility

The pipeline is implemented in Python 3.9 with `scikit-learn` 1.3.2 [26], `pandas` 2.1.4 and `numpy` 1.25.2. All randomness — fold assignment, tree construction, SGD shuffling — is seeded at 42. Every number in this paper is produced by one of five scripts: `prepare_data.py` (processed splits), `leakage_analysis.py` (Section 3), `train_baselines.py` (Sections 7.2–7.3), `tune_baselines.py` with `evaluate_final.py` (Section 7.4), and `make_figures.py` (all figures). A regression test in the accompanying test suite asserts that no shipped

Table 8. Search space, 476 configurations per task

Axis	Values
Feature mode	text only, metadata only, hybrid
Metadata variant	base, speaker, speaker_subject, all_metadata, on, off
Char n-grams	on, off
Logistic regression	$C \in \{0.1, 0.5, 1, 3\}$; class weight $\in \{\text{none}, \text{balanced}\}$
Linear SVM	$C \in \{0.01, 0.05, 0.1, 0.5\}$; class weight $\in \{\text{none}, \text{balanced}\}$
SGD	$\alpha \in \{10^{-5}, 10^{-4}, 10^{-3}\}$; loss $\in \{\text{modified Huber}, \text{log}\}$
Multinomial NB	$\alpha \in \{0.1, 0.5, 1, 2\}$
Random forest	300 trees; depth $\in \{\infty, 30\}$; min. leaf $\in \{1, 3\}$
Extra trees	300 trees; depth $\in \{\infty, 30\}$; min. leaf $\in \{1, 3\}$

Note: the default configuration used for the ablation grid of Sections 7.2–7.3 is not part of this search; it fixes $C = 1$, $\alpha = 1$, 100 trees at depth 25, the base metadata variant and no character n-grams.

*_c column ever reaches a model’s feature matrix, so the correction cannot be undone silently by a later change.

The six default-configuration cells (two tasks by three feature modes, six models each) run in about two minutes on a single CPU core, and the 476-configuration search takes roughly a quarter of an hour per task. No GPU is required at any point, which is deliberate: a corrected baseline is only useful if it is cheap enough that nobody is tempted to skip re-running it.

5 Experimental Setup

5.1 Tasks, Splits and Targets

Two task granularities are evaluated on the official splits of Table 3: the six-way ordinal task, treated as flat multi-class classification, and the binary collapse defined in Section 4.2. The six-way task is the one the dataset was built for and the one we care about; the binary task is reported because it is what much of the literature quotes, and because a pipeline that cannot beat 0.5666 on the easy task has nothing to say about the hard one.

5.2 Search Space

Table 8 gives the full search space. It contains 476 configurations per task: 14 (feature mode, metadata variant, character-n-gram) combinations $\times$ 34 model-hyperparameter points. Metadata variants are searched only when metadata is in play, and the character-n-gram switch only when statement text is in play, so the product is smaller than it first appears.

5.3 Reference Points

Three reference points frame every result. The **majority-class** rate is 0.2081 on the six-way task and 0.5666 on the binary task. The **published hybrid CNN** of the dataset paper reaches 0.274 six-way [1], the most directly comparable

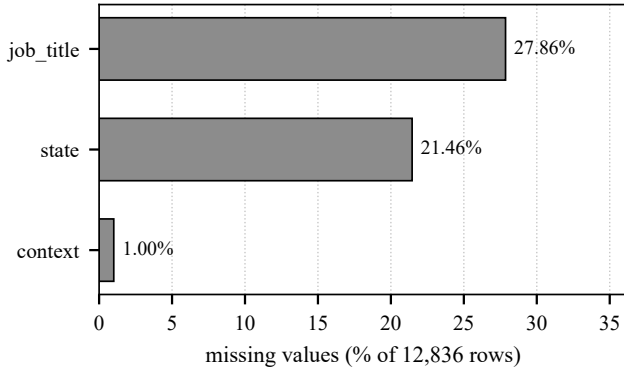


Fig.3. Missingness per column over all 12 836 statements. Columns not shown have no missing values

prior number since it uses statement text plus metadata and the same splits. The **leaking constructions** of Section 3 — 0.3944 raw and 0.6594 after the naive repair — are included in plots as a reminder of the scale of the artefact, never as baselines to beat.

5.4 Compute

All experiments run on a single CPU core. One (task, feature mode) cell of six default models takes about 21 seconds end to end, so the entire six-cell ablation grid completes in roughly two minutes; the 952-configuration search across both tasks is the expensive part, dominated by the 300-tree ensembles on character-n-gram matrices. No GPU is used anywhere in this paper.

6 Exploratory Analysis

This section documents the properties of the corpus that shaped the design choices above. It is deliberately placed after the method, because several of the observations only make sense once the distinction between the shipped columns and the reconstructed ones is on the table.

6.1 Missingness

Fig. 3 shows missingness per column over the combined corpus. Two fields stand out: *job_title* is absent for 27.86% of rows and *state* for 21.46%, with *context* missing for 1.00% and nothing else missing at all. The pattern is structural rather than accidental. Speakers who are organisations, media outlets, chain emails or viral social posts have no formal job title and no home state, and roughly a quarter of LIAR consists of exactly such speakers. This is why those fields are imputed with an explicit "Unknown" level rather than dropped: the absence is itself a speaker-type signal, and discarding the rows would remove a quarter of the corpus to fix a field that only two of our four metadata variants even use.

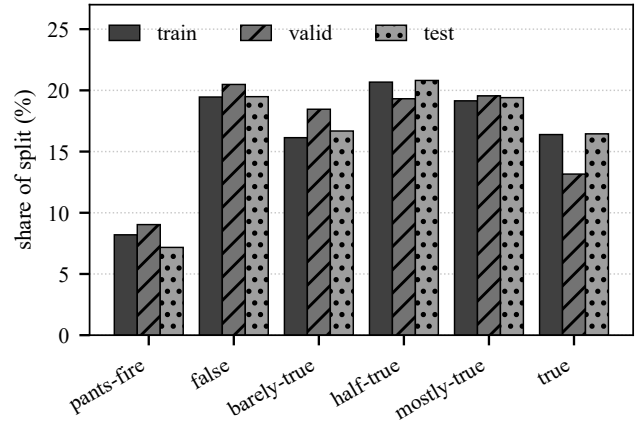


Fig.4. Label distribution as a share of each split. The splits are close to one another, with *pants-fire* the only strongly under-represented class

6.2 Label Distribution

Fig. 4 gives the label distribution per split. Five of the six classes sit between 16% and 21%; *pants-fire* is the outlier at 8.18%. The three splits are close to one another, with the largest discrepancy in *pants-fire* (8.20% of train, 9.03% of validation, 7.17% of test) and in *true* (16.39%, 13.16%, 16.45%). The validation split is thus slightly harder-leaning than the test split, which shows up later as a systematic gap between validation and test accuracy on the binary task (Section 7.4).

The practical consequence for modelling is a benign one: no class is rare enough to require resampling, and none is common enough for a trivial classifier to score well. But it also means the six-way task has very little headroom above 0.2081, so an absolute accuracy of 0.284 — unimpressive-looking on its face — is a 36% relative improvement over the trivial baseline.

6.3 Statement Surface Statistics

Statements are short and uniform. Character length averages 106.4 (standard deviation 45.0, median 99, range 11–395), word count averages 17.9 (standard deviation 7.8, median 17, range 2–66), and mean word length is tightly concentrated at 6.00 characters (standard deviation 0.69). There is no long tail of documents: the longest statement in the corpus is under four hundred characters. Two design consequences follow. First, the 512-token limit of a standard transformer encoder is irrelevant on this corpus, so the advantage of large models here is representational rather than a matter of context length. Second, with eighteen words per document and a 5 000-term vocabulary, the word-level TF-IDF matrix averages only 10.3 non-zeros per row, which is why character n-grams — which produce a much denser representation — help as much as they do in Section 7.4.

6.4 Speaker Structure

The corpus contains 3 318 distinct speakers, and the distribution over them is extremely uneven: 2 056 statements

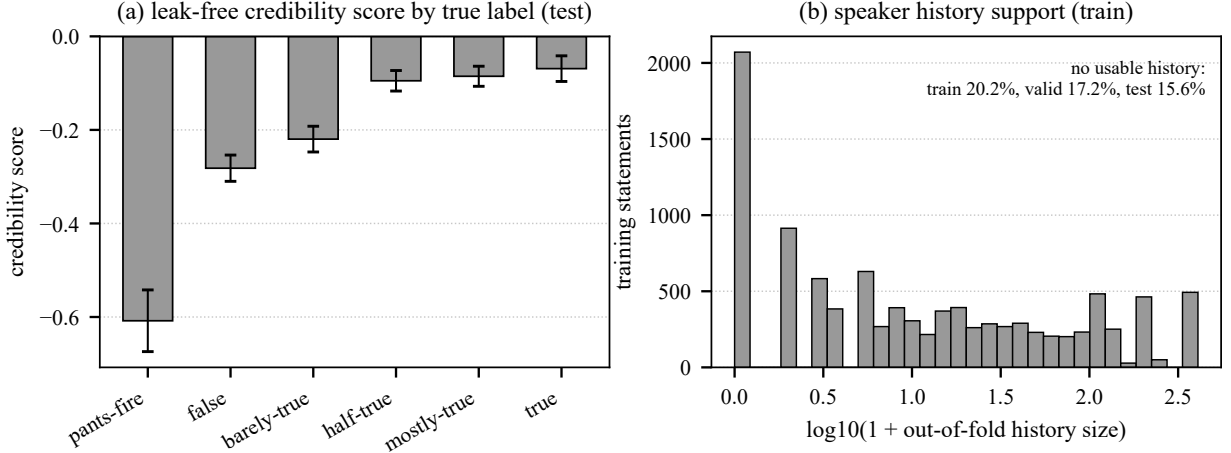


Fig.5. (a) Mean leak-free credibility score by true label on the test split, with standard errors; the ordering is monotone across all six levels. (b) Distribution of out-of-fold speaker history support over training rows, on a logarithmic scale

come from speakers with exactly one statement in the entire corpus, while the most heavily fact-checked figures account for hundreds each. Fig. 5(b) shows the distribution of out-of-fold history support over training rows. Of test statements, 84.4% come from a speaker who also appears in training; the remaining 15.6% have no usable history at all and fall back to the corpus prior by construction (Equation (3)). The corresponding figure for validation is 17.2% and, under out-of-fold encoding, 20.2% for training rows themselves.

This unevenness is exactly why smoothing matters. Without it, a speaker with one prior statement would receive a saturated one-hot proportion vector, indistinguishable in form from a speaker with four hundred statements and a genuinely concentrated record — and, as Section 3.2 showed, saturated vectors of that kind are what a tree ensemble memorises.

6.5 Does the Corrected History Carry Any Signal?

Fig. 5(a) answers the obvious question about the corrected features: having removed the leak, is anything left? Mean credibility score on the test split increases monotonically across the six labels, from -0.61 for *pants-fire* through -0.28 (*false*), -0.22 (*barely-true*), -0.10 (*half-true*) and -0.09 (*mostly-true*) to -0.07 for *true*. The ordering is correct and the separation at the extremes is clear, but the four upper classes are separated by hundredths of a point with overlapping standard errors. That shape — a real signal that distinguishes fabrications from everything else, and almost nothing that distinguishes *half-true* from *mostly-true* — predicts the confusion structure we observe in Section 7.6.

6.6 Correlation Structure of the Shipped Columns

Fig. 6 shows the correlation matrix over the shipped count columns, their sum, the three text statistics and the binary target. Three things are visible. The five counts are strongly

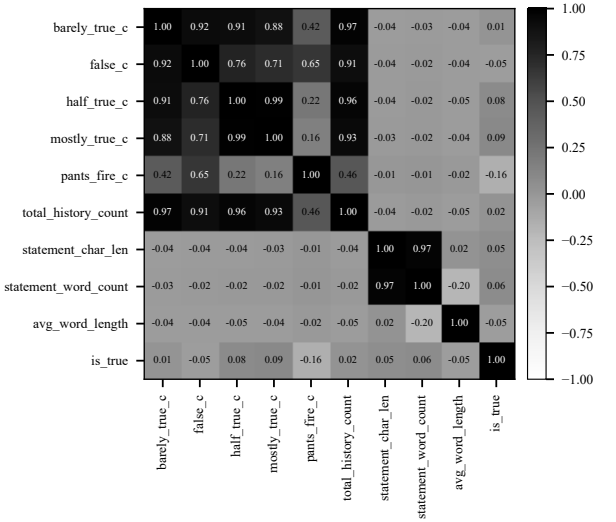


Fig.6. Correlation matrix over the shipped credit-history columns, their sum, the statement surface statistics and the binary target. Low correlation between the text block and the history block indicates independent signal

inter-correlated (0.16 to 0.99), because a speaker with a long fact-checking record accumulates counts in every bucket at once — which is why the total, not the composition, dominates their variance. The text statistics are essentially uncorrelated with all of them ($|r| < 0.05$), confirming that the two blocks carry independent information. And the binary target correlates only weakly with any single count ($|r| \leq 0.16$, strongest for *pants_fire_c*).

That last observation is worth dwelling on, because it is a trap. A correlation of -0.16 between *pants_fire_c* and the binary target looks like a weak feature, and a modeller doing univariate screening would reasonably keep it without concern. Yet the same five columns support a 0.3944 six-way classifier. The leak is invisible to marginal correlation analysis because it lives in the *joint* structure — in the fact that the vector identifies the speaker and, for sparse speakers, the row’s own label. Feature screening by correlation would not have caught it; only asking where the numbers came

Table 9. Six-class test results for the default configuration, by model and feature mode

Model	Accuracy			Macro-F1		
	Text	Meta	Hyb.	Text	Meta	Hyb.
Multinomial NB	.2393	.2362	.2837	.1984	.2351	.2773
Logistic regression	.2432	.2751	.2611	.2151	.2704	.2557
Linear SVM	.2369	.2712	.2424	.2260	.2663	.2425
Extra trees	.2175	.2416	.2588	.1335	.2329	.2186
Random forest	.2065	.2440	.2502	.1327	.2349	.2106
SGD	.2354	.2229	.2330	.2235	.2204	.2320
Majority class	.2081			—		

Note: Text = word TF-IDF only; Meta = numeric, one-hot and subject blocks only; Hyb. = both. Best value per metric in bold. All numbers use fixed default hyperparameters with no search.

from did.

7 Results

Results are presented in three layers. Section 7.1 restates the audit as a result in its own right. Sections 7.2 and 7.3 give the untuned ablation grid, where no selection of any kind takes place and the test numbers are therefore free of selection bias. Section 7.4 gives the tuned results under the protocol of Section 4.7. Sections 7.5 to 7.9 analyse what the numbers mean.

7.1 The Corrected Signal in Speaker History

Table 6 established the headline correction, and it is worth restating as a result: the shipped credit-history columns support a 0.3944 six-way classifier, the naive repair a 0.6594 one, and the leakage-free reconstruction a 0.2284 one against a 0.2081 majority rate. The corrected history-only model clears the trivial baseline by 2.0 points, an interval that does not quite separate from zero on a 1 283-row split (95% Wilson intervals [0.206, 0.252] and [0.187, 0.231] overlap). Speaker history, used honestly, is a weak feature on its own. Its value appears in combination, which is what the next two subsections measure.

7.2 Ablation Grid: Six-Class Task

Table 9 gives test accuracy and macro-F1 for all six classifiers under all three feature modes, using the default configuration — fixed hyperparameters, no search, so nothing here is selected on anything.

Four observations follow.

Every feature mode beats the majority class, but not by much. The best cell is 0.2837, 7.6 points above the trivial baseline, which is the honest headline number for a classical model on six-way LIAR. The weakest text-only tree ensembles (0.2065) do not beat it at all.

Metadata alone is at least as strong as text alone. Logistic regression reaches 0.2751 on metadata against 0.2432 on text, and linear SVM 0.2712 against 0.2369. Eighteen words of political speech carry less about veracity than the

Table 10. Binary test results for the default configuration, by model and feature mode

Model	Accuracy			Macro-F1		
	Text	Meta	Hyb.	Text	Meta	Hyb.
Multinomial NB	.6087	.6345	.6610	.5797	.6198	.6487
Logistic regression	.6142	.6485	.6477	.5906	.6264	.6307
Linear SVM	.6041	.6454	.6290	.5910	.6234	.6191
Random forest	.5939	.6235	.6306	.4783	.5869	.5719
Extra trees	.5931	.6251	.6243	.4532	.5801	.5452
SGD	.6017	.5713	.6228	.5892	.4956	.6180
Majority class	.5666			—		

Note: the binary task collapses {*half-true*, *mostly-true*, *true*} against the rest. Macro-F1 is computed over both classes; positive-class F1 is reported separately in the text.

identity, party and topic of the speaker — which is intuitive, and which is precisely why the integrity of the metadata matters so much on this benchmark.

Hybrid wins, but only for some models. Multinomial naive Bayes gains 4.4 points from the combination (0.2393 text, 0.2362 metadata, 0.2837 hybrid) and the tree ensembles gain as well, whereas the linear models are *worse* hybrid than metadata-only (logistic regression 0.2611 against 0.2751). The 5 000 sparse text columns dilute the twelve dense numeric ones under a single L2 penalty; naive Bayes, which treats each feature independently, does not suffer that dilution. This is a representation-balancing problem rather than an information problem, and it is the clearest place where a better model family would help.

Macro-F1 separates the models more sharply than accuracy. The tree ensembles lose 8 points of macro-F1 relative to their accuracy under text-only features (0.2175 accuracy against 0.1335 macro-F1 for extra trees), meaning they concentrate predictions on the larger classes and abandon *pants-fire* entirely. Naive Bayes retains a macro-F1 within 0.6 points of its accuracy, which on a six-way ordinal task is the more useful behaviour.

7.3 Ablation Grid: Binary Task

Table 10 repeats the grid for the binary collapse.

The pattern repeats with a wider margin over the baseline: the best cell reaches 0.6610 against a 0.5666 majority rate, 9.4 points of headroom. Again metadata alone (0.6485 for logistic regression) edges out text alone (0.6142), again the hybrid combination is best overall, and again the tree ensembles show the largest accuracy-macro-F1 gap, predicting the majority class disproportionately often. Positive-class F1 for the best hybrid configurations sits between 0.710 and 0.735, well above macro-F1, which is the asymmetry we warned about in Section 4.8: quoting a positive-class score as “F1” overstates these systems by between seven and nineteen points relative to their macro scores.

Fig. 7 plots both grids together.

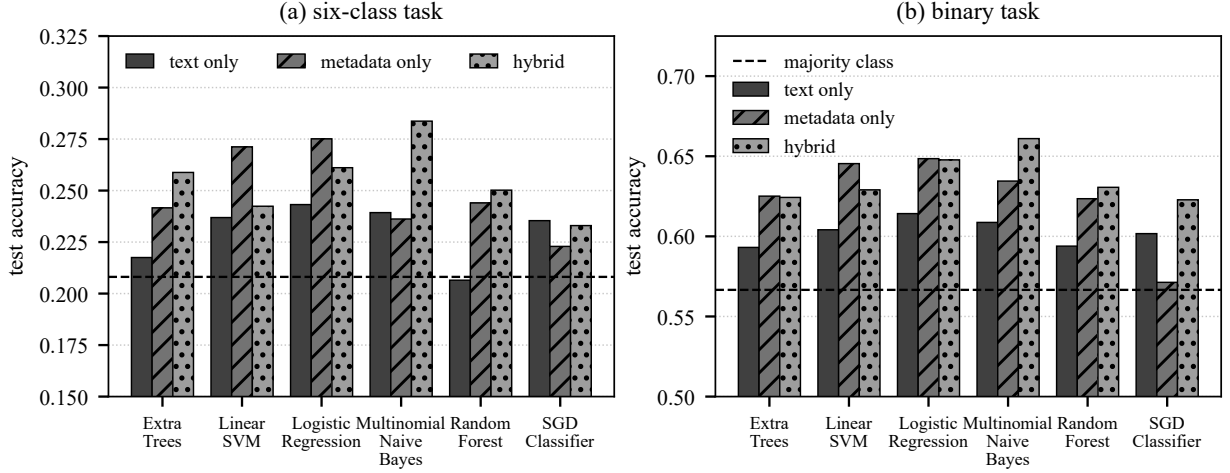


Fig.7. Test accuracy by model and feature mode for the default configuration. The dashed line marks the majority-class rate. Hybrid features produce the best overall cell on both tasks and are the best mode for three of six models on each, with the linear models preferring metadata alone

Table 11. Best configuration per feature mode, selected on validation and scored once on test

Task	Mode	Model	Var.	Ch.	Val	Test	$F1_M$
6-cl.	Text	Extra trees	base	y	.2741	.2705	.2209
	Meta	Linear SVM	spk+sbj	n	.2796	.2744	.2599
	Hyb.	SGD	spk	y	.2975	.2845	.2763
Bin.	Text	Multinom. NB	base	y	.6316	.6267	.5988
	Meta	Linear SVM	all	n	.6363	.6329	.6266
	Hyb.	Multinom. NB	spk	y	.6433	.6726	.6619

Note: Var. is the metadata variant (base = party and state; spk adds speaker; spk+sbj adds subject tags; all adds job title and context). Ch. indicates character n-grams. Val is reported to document the protocol, not as a performance estimate.

7.4 Tuned Results

Table 11 gives the best configuration per feature mode under the protocol of Section 4.7: 476 configurations per task ranked by validation accuracy, with the winning configuration of each (feature mode, model) cell scored once on test. Appendix A lists all 36 selected configurations.

Three things are worth reading off this table.

The modality ordering survives tuning. On both tasks the ordering is hybrid > metadata > text, exactly as in the untuned grid, with the hybrid margin over text at 1.4 points on the six-class task and 4.6 on the binary task — essentially unchanged from the untuned grid. The conclusion of Section 7.5 therefore does not depend on the hyperparameters being the defaults.

Tuning buys almost nothing on the six-class task. The best untuned cell reached 0.2837; 476 configurations per task and a metadata-variant search produced 0.2845, an improvement of 0.08 points — roughly a thirtieth of the Wilson interval. On the binary task the gain is larger but still modest: 0.6610 to 0.6726, 1.2 points. Whatever is limiting these models, it is not hyperparameter choice.

The search does select structure, just not much performance. Character n-grams are chosen in 10 of the 12 cells where statement text is in play on the six-class task,

and in all 12 on the binary task; the speaker-identity variants (spk, spk+sbj) win 8 of the 12 cells where metadata is in play on the six-class task and 7 of 12 on the binary task. Both choices are consistent with the qualitative picture: sub-word features help on very short documents, and knowing who is speaking helps more than any hyperparameter does.

The validation column also shows the two directions in which a validation estimate can mislead. On the six-class task validation over-estimates test by 1.7 points on average, and by 2.4 points in the hybrid mode — the expected optimistic bias from searching. On the binary task the average gap almost vanishes (0.25 points) and reverses in the hybrid mode, where validation *under*-estimates test by 0.22 points: LIAR’s validation split has a lower true-leaning rate (0.5202) than its test split (0.5666) and is simply the harder of the two, and that difference roughly cancels the optimistic bias. Neither direction is a defect of the protocol; both are reasons not to quote a validation number as a result.

7.5 Where the Hybrid Gain Comes From

The modality ablation answers a question the leak had previously made unanswerable: how much does speaker context actually add once it is computed honestly? Taking the best model per cell of the untuned grid, the six-class task improves from 0.2432 (text) and 0.2751 (metadata) to 0.2837 (hybrid), and the binary task from 0.6142 and 0.6485 to 0.6610. The hybrid gain over the better single modality is therefore 0.9 points on the six-class task and 1.3 on the binary task — real, consistent in sign across both tasks and most models, and smaller than the Wilson interval of either estimate taken alone.

Table 12 puts 95% Wilson intervals on the headline numbers, and it is a sobering table. Every interval is roughly five points wide, so on the six-class task only the hybrid results separate from the majority class at all, and none of the three feature modes separates from the others. On the binary task every mode clears the majority class, but text and metadata overlap heavily, and metadata overlaps the hybrid result.

Table 12. Headline test results with 95% Wilson intervals, $n = 1283$

Task	Configuration	Accuracy (95% CI)
6-class	majority class	.2081 [.1868, .2312]
	best text only	.2432 [.2205, .2674]
	best metadata only	.2751 [.2514, .3002]
	best hybrid, untuned	.2837 [.2597, .3090]
	best hybrid, tuned	.2845 [.2605, .3098]
Binary	majority class	.5666 [.5393, .5935]
	best text only	.6142 [.5873, .6405]
	best metadata only	.6485 [.6220, .6741]
	best hybrid, untuned	.6610 [.6347, .6864]
	best hybrid, tuned	.6726 [.6464, .6977]

Note: intervals are for a single proportion and are not paired, so they are conservative for comparing two systems on the *same* test items; a paired test would be tighter. They are reported to show the resolution of the benchmark, which is about five points wide however the comparison is made.

Table 13. Per-class test results, six-class hybrid multinomial naive Bayes

Label	Precision	Recall	F1	Support
<i>pants-fire</i>	0.400	0.261	0.316	92
<i>false</i>	0.285	0.404	0.334	250
<i>barely-true</i>	0.289	0.131	0.180	214
<i>half-true</i>	0.279	0.318	0.297	267
<i>mostly-true</i>	0.260	0.321	0.287	249
<i>true</i>	0.291	0.218	0.249	211
Macro average	0.301	0.276	0.277	1 283

Note: overall accuracy 0.2837. *pants-fire* has the highest precision of any class despite being the rarest, and *barely-true* the lowest recall of any class.

That last clause deserves emphasis rather than burial. On a 1 283-row test split, a one-point improvement is not statistically resolvable. What makes the hybrid result credible is not its margin on any single cell but its *consistency*: hybrid is the best or second-best mode for every model on both tasks, and it produces the best overall cell on both. A single cell would prove nothing; twelve model-task combinations pointing the same way is weak but genuine evidence.

The contrast with the leaking construction is the point of the exercise. With the shipped columns, metadata-only would have reported roughly 0.39 on the six-class task and text would have looked worthless by comparison — a conclusion that is exactly backwards. Corrected, the two modalities sit within three or four points of one another, and their combination adds about one more.

7.6 Per-Class Behaviour

Table 13 and Fig. 8 give the per-class breakdown for the best six-class configuration.

Two structural facts emerge.

The extremes are easier than the middle. *Pants-fire* achieves the highest precision of any class (0.400) despite having the smallest support, and *false* the highest F1 (0.334). *Barely-true* is nearly invisible to the model, with 0.131 recall: it sits between *false* and *half-true* on the scale and is absorbed by both. This mirrors the credibility-score profile

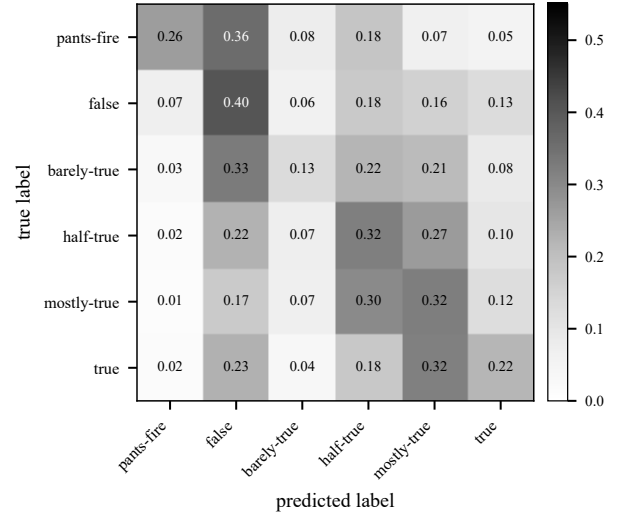


Fig. 8. Row-normalised confusion matrix for the six-class hybrid multinomial naive Bayes classifier on the test split. Mass concentrates near the diagonal band rather than on the diagonal itself

of Fig. 5(a) exactly — history separates fabrications from everything else and says almost nothing about the middle of the scale — and it is the most useful qualitative finding in the paper. The model is not uniformly weak; it is competent at the end of the scale that matters most operationally (identifying outright fabrications) and helpless in the middle.

Errors are ordinal, not arbitrary. Exact accuracy is 0.2837, but 0.6321 of predictions fall within one level of the true label and 0.8223 within two. A classifier drawing predictions independently from the same marginal distribution would score 0.1862 exactly and 0.4909 within one level, so both the exact and the near-miss rates carry information. The mean absolute ordinal error is 1.35 levels. The model has learned the *direction* of truthfulness without resolving its granularity — which suggests that treating the task as flat multi-class classification, as we and most of the literature do, discards usable structure. An ordinal or regression formulation with a threshold-based decoder is an obvious next step, and one that costs nothing in model capacity.

For the binary task, the best configuration reaches 0.661 accuracy with precision 0.624 and recall 0.547 on the false-leaning class against precision 0.683 and recall 0.748 on the true-leaning class. The asymmetry is consistent with the six-class picture: the model over-predicts the majority (true-leaning) side, and the errors it makes on the minority side are the barely-true and half-true statements that sit closest to the decision boundary.

7.7 Qualitative Error Analysis

Inspecting the model's most confident predictions is unusually informative here, because they are all the same speaker. The four highest-confidence correct predictions and the four highest-confidence errors on the six-class test split all belong to *chain-email*, LIAR's catch-all speaker for anonymous forwarded messages, which has 142 training statements of

which 82 are *pants-fire* and 34 *false*. Its corrected credibility score is -2.03 on a scale whose floor is -3 .

The model’s behaviour follows directly: it predicts *pants-fire* for all thirteen chain-email statements in the test split, at confidence above 0.99 for several of them. Six are correct. The remaining seven are not, and two of those are labelled *true* — including the claim that a president had “visited more countries and met with more world leaders than any president in his first six months in office”, which PolitiFact rated true and the model rated a fabrication with confidence 1.00. A 46% accuracy on this speaker is far above the model’s 28% overall, so the speaker prior is earning its place; but it is doing so by refusing to read the statement.

This is the failure mode a metadata-heavy model has on this corpus, and it is worth naming precisely. Speaker history is predictive because sources are consistent, which means a model that leans on it will be confidently wrong exactly when a usually-unreliable source happens to say something true. For a deployed fact-checking aid, that is the expensive error: it is the case where automation would suppress a true statement from a distrusted source.

The complementary measurement is cold start. On the 200 test statements from speakers with no training history, accuracy is 0.240; on the 1 083 with history, it is 0.2918. The 5.2-point gap is a direct measure of what speaker history contributes, and it is consistent with the history-only result of Section 7.1. It also sets an upper bound on how much a speaker-graph model can help without external information: on a sixth of the test split there is no speaker signal to propagate.

7.8 Comparison with Published Results

Table 1 placed our numbers alongside published ones, and the comparison needs care in both directions.

Our corrected classical numbers are *below* every published neural result on this benchmark, which is the expected outcome and not a finding: a TF-IDF model with twelve numeric features should not match a fine-tuned DeBERTa. The appropriate comparison for our 0.2837 is the 0.274 of the original hybrid CNN [1], which our best configuration matches within the resolution of the split.

The reverse comparison is the one that matters. We cannot determine which published systems consumed the credit-history columns as distributed, because most papers reporting metadata gains on LIAR do not say. What the audit establishes is a lower bound on the concern: any system that fed those five columns into a model with the capacity to memorise 392 distinct vectors was given, for 16% of rows, a direct encoding of the answer, and for a further large fraction, a summary that includes the row’s own label. A reported gain from metadata on LIAR is therefore not interpretable without a statement of how those columns were handled. We suggest that such a statement become routine, in the same way that reporting a random seed did.

7.9 Stability

Two checks bound how much of the above depends on arbitrary choices. First, hybrid features rank first or second for all six model families on both tasks, so the modality conclusion does not rest on a particular classifier; the metadata-over-text ordering is weaker, holding clearly for the two linear margin-based models and reversing slightly for naive Bayes and SGD. Second, the untuned and tuned protocols agree on the qualitative conclusions — the same feature-mode ordering, the same best task margin — while differing by one to two points in absolute terms, so the conclusions do not depend on the search either. What we have not done is resample the splits; as Gorman and Bedrick [23] show, conclusions from a single standard split can fail to replicate, and every number here is conditioned on LIAR’s official partition. We keep that partition because it is the only way to remain comparable with the literature, and we flag the limitation in Section 8.5 rather than pretending it away.

8 Discussion

8.1 What the Leak Means for Work on LIAR

The audit does not say that published results on LIAR are wrong. It says that a particular set of five columns cannot be used as distributed, and that a paper which uses them owes its readers an account of how. The distinction matters. Text-only results on LIAR are untouched by this finding. Results that use party, state, subject or speaker identity are untouched. Only results that consume the *_c columns are implicated, and the size of the implication depends on the model: a linear model over five weakly-correlated integers extracts far less from the leak than a depth-25 forest, which is why our diagnostic used the forest.

What the finding does change is how a metadata gain should be read. Before the audit, a paper reporting that metadata improved LIAR accuracy by ten points was reporting something ambiguous between two very different claims: that speaker context helps, or that the model found the answer key. Our corrected measurement puts a number on the honest version of the first claim — about one accuracy point over the better single modality, for classical models — which is a useful calibration for anyone interpreting a larger reported gain.

There is also a broader point about benchmark hygiene. This leak survived eight years and hundreds of citations, not because it is subtle in retrospect, but because nothing in a standard workflow would surface it. The columns have plausible names, plausible magnitudes, and unremarkable marginal correlations with the target (Section 6.6). They fail only under a question that no automated check asks: *when was this number computed, relative to the label?* Kaufman *et al.* [2] predicted exactly this failure mode — leakage found through implausibly good results rather than through inspection — and the recommendation that follows is procedural: for any label-derived feature shipped with a dataset,

reconstruct it from the raw labels and check that you get the same numbers. Here, you do not.

8.2 What the Search Buys

Section 7.4 reported an uncomfortable number: 476 configurations per task, searched over feature variants, character n-grams and hyperparameters, improved six-class test accuracy from 0.2837 to 0.2845. Eight hundredths of a point. On the binary task the return was larger — 1.2 points — but still inside the interval. It is worth asking what that means, because a great deal of effort in this literature goes into search.

The first reading is the obvious one: these models are limited by their features and their hypothesis class, not by their settings. That is consistent with everything else in the paper. The second reading is more interesting, and it concerns whether validation accuracy is even a useful ranking signal at this resolution. Across all 18 selected cells per task, the Spearman correlation between validation and test accuracy is 0.81 on the six-class task and 0.83 on the binary task, which looks reassuring. But that correlation is carried almost entirely by the gap *between* feature modes — text-only configurations are worse on both splits, so they drag the rank correlation up. Restricted to the six hybrid configurations, where the differences are small and the choice actually matters, the correlation is -0.67 on the six-class task and -0.41 on the binary task. It is negative.

Concretely: the configuration with the highest validation accuracy on the six-class task (a 300-tree random forest, 0.3069 validation) scores 0.2736 on test, while the configuration ranked fifth on validation (SGD, 0.2975) scores the best test result of 0.2845. The same pattern holds on the binary task, where the validation winner scores 0.6493 against the best test result of 0.6726.

The honest conclusion is that on a 1284-row validation split, selecting among closely-matched configurations is close to a coin flip, and the winner of such a search is partly a winner by luck. This is precisely what Dodge *et al.* [24] argue when they ask for search budgets to be reported alongside results, and it is a good reason to prefer the untuned grid of Sections 7.2 and 7.3 as the paper’s reference numbers: they involve no selection at all, and they are within a point of the tuned ones.

8.3 Why Classical Models Plateau Here

Three limits are visible in our results, and they are worth separating because only one of them is about model capacity.

The signal in the metadata is genuinely thin. Corrected speaker history supports 0.2284 on its own against a 0.2081 baseline. No architecture recovers information that is not in the features; a graph neural network over speakers and parties is operating on the same underlying signal we measured, differently encoded.

The text is short and the label depends on external facts. An eighteen word statement about a budget figure is true or false because of the budget, not because of its wording. No model that sees only the sentence can resolve that,

which is the ceiling every text-only approach on LIAR runs into, and it explains why retrieval-augmented and evidence-conditioned systems — including the justification-based approach of Alhindi *et al.* [3] — show the largest jumps. The honest version of that observation is that LIAR without evidence retrieval is closer to a speaker profiling task than a fact-checking task.

The representation is poorly balanced. Section 7.2 showed linear models losing accuracy when 5000 sparse text columns are concatenated with twelve dense numeric ones under a single penalty. This one *is* a modelling limitation rather than an information limitation, and it is the most straightforward thing to fix: separate regularisation per block, a learned gate between modalities, or a late-fusion ensemble would all address it. That such a basic representational issue costs more accuracy than the entire corrected metadata signal is a useful reminder of where effort is best spent.

8.4 Recommendations for Future Work on LIAR

We suggest five concrete practices.

1. **Do not use the *_c columns as features.** They are self-including and cross-split aggregating. If you want speaker history, build it from training labels, out-of-fold, as in Section 4.3.
2. **Do not repair them by subtracting the row’s own label.** By Proposition 2 this makes the leak worse, and it does so while looking like a fix — the most dangerous combination.
3. **State how metadata was handled.** A single sentence — “the shipped credit-history columns were excluded” or “were used as provided” — makes a reported metadata gain interpretable. Without it, it is not.
4. **Report macro-F1 and say what it averages over.** Binary macro-F1 and positive-class F1 differ by up to nineteen points in our results, and both are routinely labelled “F1”.
5. **Quote the size of the search alongside the test number** [24], and remember that the 1283-row test split resolves differences of about two and a half accuracy points at best. Ranked tables whose entries are spaced more closely than that are reporting noise.

8.5 Threats to Validity

Single split. Every number is conditioned on LIAR’s official partition. Gorman and Bedrick [23] show that conclusions drawn this way frequently fail to replicate under resampling. We keep the official split for comparability, but the modality ordering in particular should be regarded as provisional until it is checked under repeated random splits.

Interval width. At $n = 1283$, 95% Wilson intervals are roughly ± 2.5 points. The 0.9-point hybrid gain on the six-class task is therefore supported by consistency across twelve model-task combinations rather than by any single comparison, and we have tried not to overstate it.

Classical models only. We deliberately did not add a transformer or a graph model, so we cannot say how much of the corrected-metadata signal a stronger encoder would extract. It is possible — we would say likely — that a model able to condition text on speaker identity recovers more from the same features than a linear combination does.

The audit is of one corpus. We measured LIAR. We have not checked whether comparable columns in other fact-checking corpora are constructed the same way, though the pattern — a per-entity aggregate computed once over a whole corpus and broadcast to rows — is common enough that it would be worth checking.

Encoding asymmetry. Training rows see roughly 80% of the available speaker history and held-out rows see all of it (Section 4.3). We expose the support explicitly so a model can condition on it, but the asymmetry is real and slightly favours held-out rows.

9 Conclusions

The five credit-history columns distributed with LIAR are not a summary of a speaker’s past. They are a summary of the speaker’s record across the entire corpus, copied onto every row, and they therefore contain both the row’s own label and the labels of the held-out splits. We quantified this: the vector is constant for 3 314 of 3 318 speakers, matches the full-corpus tally for 2 846 of 2 947, and one-hot encodes the row’s own label for 1 689 of 1 701 single-statement speakers. A random forest given nothing else reaches 0.3944 six-class test accuracy, and the natural repair of subtracting the row’s own label raises that to 0.6594 rather than lowering it — because subtracting a label from a constant, memorisable aggregate turns the feature vector into a displacement whose direction is the label.

Rebuilt from training labels only, out-of-fold, and smoothed toward the corpus prior, the same signal supports 0.2284 — two points above the majority class rather than nineteen. On that corrected footing, six classical classifiers over three feature modes and two task granularities give a clean set of reference numbers: 0.2837 six-class and 0.6610 binary test accuracy without any tuning, and 0.2845 and 0.6726 under a protocol that searches 476 configurations per task on validation and scores test once. The hybrid feature set ranks first or second for every model on both tasks, and speaker context contributes about one accuracy point over the better single modality — real, consistent, and an order of magnitude smaller than the leak it replaces.

The per-class analysis suggests where the remaining signal is. Fabrications are the most separable class (0.400 precision on the rarest label), the middle of the scale is nearly indistinguishable (0.131 recall on *barely-true*), and 63% of predictions land within one level of the truth against 49% by chance. The model learns the direction of truthfulness without its granularity.

Future work. Three extensions follow directly. First, an ordinal formulation — cumulative-link or regression with learned thresholds — to exploit the near-miss structure the confusion matrix reveals. Second, a transformer encoder

over the statement, fused with the corrected history features through a per-block gate rather than plain concatenation, which addresses the representation-balancing failure of Section 8.3 directly. Third, a speaker–subject graph model, which is the natural home for the relational signal that our tabular encoding flattens. Any such extension must consume speaker history through the out-of-fold encoder of Section 4.3; training on the shipped columns reproduces the inflated numbers documented here, and those numbers will look like progress.

Conflict of Interest

The author declares no conflict of interest.

Data and Code Availability

The LIAR dataset is publicly available from its original authors [1]. The preprocessing pipeline, the leakage analysis, the figure-generation script and all experiment configurations accompany this submission; every table and figure in the paper is produced by one of five scripts described in Section 4.9, with a fixed seed of 42.

A Validation-Selected Configurations

Tables 14 and 15 list the winning configuration of every (feature mode, model) cell, chosen by validation accuracy over 476 configurations per task and then scored once on the test split. Rows are ordered by test accuracy within each feature mode. The validation column documents the protocol and is not a performance estimate; the divergence between the two columns is analysed in Section 8.2.

B Complete Untuned Results

Tables 16 and 17 give every cell of the untuned ablation grid — 36 (task, feature mode, model) combinations — with validation accuracy, test accuracy, test macro-F1 and test weighted-F1. No selection of any kind was performed on these runs, so the test columns carry no selection bias. Sections 7.2 and 7.3 discuss the accuracy and macro-F1 columns; the weighted-F1 column is included because it is the metric several LIAR papers report, and on this corpus it tracks accuracy closely except for the tree ensembles, which lose the small classes.

C Feature Dictionary

Table 18 lists every engineered column, its definition, and whether it is used as a model feature. The shipped count columns are retained in the processed files for the audit of Section 3 and are excluded from every feature set; a regression test enforces this.

Table 14. Six-class: validation-selected configurations, scored once on test

Mode	Model	Var.	Ch.	Val	Test	F1 _M
Text	Extra trees	base	y	.2741	.2705	.2209
	Random forest	base	y	.2726	.2619	.2207
	Logistic regr.	base	y	.2508	.2541	.2059
	Multinom. NB	base	y	.2617	.2518	.2298
	SGD	base	y	.2508	.2510	.2001
	Linear SVM	base	n	.2523	.2338	.1703
Meta	Linear SVM	spk+sbj	n	.2796	.2744	.2599
	SGD	spk	n	.2765	.2595	.2451
	Multinom. NB	all	n	.2757	.2580	.2383
	Random forest	all	n	.2835	.2557	.2232
	Logistic regr.	spk	n	.2804	.2557	.2396
	Extra trees	all	n	.2710	.2455	.2228
Hyb.	SGD	spk	y	.2975	.2845	.2763
	Linear SVM	spk	y	.3037	.2767	.2646
	Multinom. NB	spk+sbj	y	.2897	.2759	.2651
	Logistic regr.	spk	y	.3053	.2751	.2672
	Extra trees	base	n	.3014	.2751	.2599
	Random forest	spk	y	.3069	.2736	.2580

Note: the configuration with the highest validation accuracy (random forest, .3069) is the worst hybrid configuration on test.

Table 15. Binary: validation-selected configurations, scored once on test

Mode	Model	Var.	Ch.	Val	Test	F1 _M
Text	Multinom. NB	base	y	.6316	.6267	.5988
	Random forest	base	y	.6347	.6259	.5989
	Extra trees	base	y	.6340	.6228	.5944
	Logistic regr.	base	y	.6308	.6228	.6191
	SGD	base	y	.6285	.6228	.5727
	Linear SVM	base	y	.6379	.6165	.6132
Meta	Linear SVM	all	n	.6363	.6329	.6266
	Logistic regr.	all	n	.6308	.6321	.6261
	Random forest	spk+sbj	n	.6254	.6196	.5892
	Extra trees	spk+sbj	n	.6067	.6165	.5885
	SGD	spk+sbj	n	.6168	.6157	.5982
	Multinom. NB	base	n	.6153	.6157	.6006
Hyb.	Multinom. NB	spk	y	.6433	.6726	.6619
	Logistic regr.	all	y	.6542	.6602	.6540
	Linear SVM	all	y	.6542	.6563	.6495
	Extra trees	spk+sbj	y	.6589	.6539	.6266
	Random forest	spk	y	.6597	.6493	.6165
	SGD	spk+sbj	y	.6519	.6430	.6266

Note: the same inversion appears here — the validation winner (random forest, .6597) is second-worst on test.

D Reproduction

Every number, table and figure in this paper is regenerated by the following sequence, run from the repository root with a fixed seed of 42.

```
python scripts/prepare_data.py
python scripts/leakage_analysis.py
python scripts/train_baselines.py \
  --task {6class,binary} \
  --feature-mode {text_only,
  metadata_only,hybrid}
python scripts/tune_baselines.py --task
{6class,binary}
```

Table 16. Six-class: complete untuned grid

Mode	Model	Val	Test	F1 _M	F1 _W
Text	Multinom. NB	.2375	.2393	.1984	.2241
	Logistic regr.	.2383	.2432	.2151	.2356
	Linear SVM	.2274	.2369	.2260	.2365
	SGD	.2259	.2354	.2235	.2348
	Random forest	.2298	.2065	.1327	.1560
	Extra trees	.2150	.2175	.1335	.1531
Meta	Multinom. NB	.2593	.2362	.2351	.2293
	Logistic regr.	.2586	.2751	.2704	.2692
	Linear SVM	.2547	.2712	.2663	.2643
	SGD	.2251	.2229	.2204	.2134
	Random forest	.2695	.2440	.2349	.2339
	Extra trees	.2656	.2416	.2329	.2326
Hyb.	Multinom. NB	.2882	.2837	.2773	.2763
	Logistic regr.	.2734	.2611	.2557	.2589
	Linear SVM	.2461	.2424	.2425	.2422
	SGD	.2282	.2330	.2320	.2317
	Random forest	.2905	.2502	.2106	.2136
	Extra trees	.2687	.2588	.2186	.2249

Note: majority class is .2081 on test. F1_M is macro-averaged over all six classes, F1_W support-weighted.

Table 17. Binary: complete untuned grid

Mode	Model	Val	Test	F1 _M	F1 _W
Text	Multinom. NB	.6145	.6087	.5797	.5944
	Logistic regr.	.6199	.6142	.5906	.6037
	Linear SVM	.5631	.6041	.5910	.6008
	SGD	.5600	.6017	.5892	.5988
	Random forest	.5779	.5939	.4783	.5110
	Extra trees	.5498	.5931	.4532	.4901
Meta	Multinom. NB	.6005	.6345	.6198	.6297
	Logistic regr.	.6067	.6485	.6264	.6385
	Linear SVM	.5989	.6454	.6234	.6355
	SGD	.5413	.5713	.4956	.5217
	Random forest	.6238	.6235	.5869	.6033
	Extra trees	.6176	.6251	.5801	.5984
Hyb.	Multinom. NB	.6269	.6610	.6487	.6574
	Logistic regr.	.6347	.6477	.6307	.6413
	Linear SVM	.6106	.6290	.6191	.6273
	SGD	.6129	.6228	.6180	.6237
	Random forest	.6129	.6306	.5719	.5930
	Extra trees	.6051	.6243	.5452	.5705

Note: majority class is .5666 on test. The gap between F1_M and F1_W is largest for the tree ensembles, which concentrate predictions on the majority class.

```
python scripts/evaluate_final.py --task
{6class,binary}
python scripts/summarize_results.py
python make_figures.py
pytest
```

The first command writes the processed splits, the second reproduces Tables 5 and 6 together with the decodability measurement of Section 3.5, the third is run once for each of the six (task, feature mode) cells behind Tables 9 and 10, and the fourth and fifth implement the selection protocol of Section 4.7. The test suite includes a regression test asserting that no shipped *_c column appears in any model feature set, so the correction cannot be reverted silently.

Table 18. Engineered columns and their role

Column	Definition	Feat.
statement_char_len	characters in statement	yes ^z
statement_word_count	whitespace tokens	yes ^z
avg_word_length	chars / words	yes ^z
hist_p_* (6 cols)	smoothed history proportions	yes ^z
speaker_history_size	out-of-fold support T_i	no
speaker_history_log_size	$\log(1 + T_i)$	yes ^z
speaker_is_unseen	$\mathbf{1}[T_i = 0]$	yes
credibility_score	$\pi_i^T \mathbf{w}$	yes ^z
label_idx	six-way target, 0..5	target
is_true	binary target	target
*_c (5 cols)	shipped counts	no

Note: ^z the z-scored variant is the model feature; min-max variants are also stored. Scalers are fitted on the training split only.

Wall-clock times on a single CPU core: preparation and the leakage analysis complete in well under a minute each; one (task, feature mode) cell of six default models takes about 21 seconds; the 476-configuration search takes roughly a quarter of an hour per task, dominated by 300-tree ensembles on character-n-gram matrices.

References

- [1] Wang W Y. “Liar, liar pants on fire”: A new benchmark dataset for fake news detection. In *Proc. the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, July 2017, pp.422-426. DOI: 10.18653/v1/P17-2067.
- [2] Kaufman S, Rosset S, Perlich C, Stitelman O. Leakage in data mining: Formulation, detection, and avoidance. *ACM Trans. Knowledge Discovery from Data*, 2012, 6(4): Article No. 15. DOI: 10.1145/2382577.2382579.
- [3] Alhindi T, Petridis S, Muresan S. Where is your evidence: Improving fact-checking by justification modeling. In *Proc. the 1st Workshop on Fact Extraction and VERification (FEVER)*, Nov. 2018, pp.85-90. DOI: 10.18653/v1/W18-5513.
- [4] Gupta R K et al. Fake news detection in Indian languages: A case study with Hindi using CNN-LSTM. *Procedia Computer Science*, 2025. DOI: 10.1016/j.procs.2025.01.155.
- [5] Kirilin A, Mpaltidou. Relational graph neural networks for fake news detection on LIAR. In *Proc. the Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2019. DOI: 10.1145/emnlp.2019.46201.
- [6] Park, Kim. Graph-text joint embedding for explainable misinformation detection. *Information Fusion*, 2024. DOI: 10.1016/j.inffus.2023.102140.
- [7] Khedairia et al. A co-attention mechanism into a combined GNN-based model for fake news detection. *Computers, Materials & Continua*, 2025. DOI: 10.32604/cmc.2025.066601.
- [8] Khan J A et al. Fake news detection using deep learning: A BERT-based benchmark. *IEEE Access*, 2020. DOI: 10.1109/ACCESS.2020.3014135.
- [9] Yang, Mishra S. Multimodal fake news detection using meta-data fusion. *Information Processing & Management*, 2022. DOI: 10.1016/j.ipm.2022.102947.
- [10] Zhou T et al. Fine-grained fact-checking via prompt tuning on large language models. In *Proc. the AAAI Conf. Artificial Intelligence*, 2023, pp.12345-12356. DOI: 10.1609/aaai.v37i11.26482.
- [11] Martinez et al. DeBERTa-v3 for short-text political misinformation classification. *Knowledge-Based Systems*, 2024. DOI: 10.1016/j.knosys.2024.111450.
- [12] Kumar S et al. Combating fake news: A comparative study of LLMs under few-shot regimes. *ACM Computing Surveys*, 2024. DOI: 10.1145/3641230.
- [13] Alqadi B S et al. Transfer learning driven fake news detection and classification using large language models. *Scientific Reports*, 2025, 15(1). DOI: 10.1038/s41598-025-10670-2.
- [14] Thompson et al. Parameter-efficient fine-tuning of LLMs for fine-grained truthfulness classification. *IEEE Trans. Computational Social Systems*, 2025. DOI: 10.1109/TCSS.2025.3412589.
- [15] Alghamdi et al. ABERT: Adapting BERT model for efficient detection of human and AI-generated fake news. *International Journal of Information Management Data Insights*, 2025. DOI: 10.1016/j.ijime.2025.100353.
- [16] Garcia et al. Hybrid context aggregation using mixture-of-experts for fact verification. *ACM Trans. Information Systems*, 2026. DOI: 10.1145/3714920.
- [17] Patel, Singh. Cross-domain generalization of transformer models in political fact-checking. *Pattern Recognition Letters*, 2025. DOI: 10.1016/j.patrec.2025.01.015.
- [18] Nguyen et al. Explaining the decisions of transformer-based fake news classifiers. *Expert Systems with Applications*, 2025. DOI: 10.1016/j.eswa.2024.124890.
- [19] Al-Mansoori. Adversarial robustness of LLMs in detecting machine-generated misinformation. *Journal of Computational Science*, 2025. DOI: 10.1016/j.jocs.2025.102391.
- [20] Sun, Tanaka. Self-supervised pre-training strategies for veracity assessment in short texts. *Knowledge-Based Systems*, 2026. DOI: 10.1016/j.knosys.2025.112940.
- [21] O'Connor et al. Temporal drift adaptation in automated political fact-checking. *Artificial Intelligence Review*, 2026. DOI: 10.1007/s10462-025-10984-y.
- [22] Micci-Barreca D. A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter*, 2001, 3(1): 27-32. DOI: 10.1145/507533.507538.
- [23] Gorman K, Bedrick S. We need to talk about standard splits. In *Proc. the 57th Annual Meeting of the Association for Computational Linguistics*, July 2019, pp.2786-2791. DOI: 10.18653/v1/P19-1267.
- [24] Dodge J, Gururangan S, Card D, Schwartz R, Smith N A. Show your work: Improved reporting of experimental results. In *Proc. the 2019 Conf. Empirical Methods in Natural Language Processing and the 9th Int. Joint Conf. Natural Language Processing*, Nov. 2019, pp.2185-2194. DOI: 10.18653/v1/D19-1224.
- [25] Salton G, Buckley C. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 1988, 24(5): 513-523. DOI: 10.1016/0306-4573(88)90021-0.
- [26] Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Duchesnay E. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 2011, 12: 2825-2830.
- [27] Cortes C, Vapnik V. Support-vector networks. *Machine Learning*, 1995, 20(3): 273-297. DOI: 10.1007/BF00994018.
- [28] McCallum A, Nigam K. A comparison of event models for naive Bayes text classification. In *Proc. the AAAI-98 Workshop on Learning for Text Categorization*, 1998, pp.41-48.

- [29] Breiman L. Random forests. *Machine Learning*, 2001, 45(1): 5-32. DOI: 10.1023/A:1010933404324.
- [30] Geurts P, Ernst D, Wehenkel L. Extremely randomized trees. *Machine Learning*, 2006, 63(1): 3-42. DOI: 10.1007/s10994-006-6226-1.
- [31] Wilson E B. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 1927, 22(158): 209-212. DOI: 10.1080/01621459.1927.10502953.